

山东大学



网络安全创新创业实践

SM3 软件实现与优化

姓名：杨阳 学号：202300460010

姓名：孙琬竺 学号：202300460020

姓名：胡子玮 学号：202300460032

姓名：熊芷萱 学号：202300460163

2026 年 7 月 28 日

SM3 软件实现与优化

目录

1	实验背景	1
1.1	SM3 算法概述	1
1.2	SIMD 优化背景	1
2	实现方式	2
2.1	项目结构	2
2.2	混合优化策略	2
2.3	AVX2 SIMD 实现细节	2
2.4	ARM64 NEON 实现细节	3
2.5	关键技术处理	4
3	实现过程	4
3.1	开发步骤	4
3.2	测试结果	7
4	实验中遇到的问题与解决	10
4.1	循环移位未定义行为	10
4.2	不同消息长度导致 SIMD 结果不匹配	10
4.3	测试向量中 hex 字符串长度计算	10
5	总结	11
	参考文献	11
6	分工情况	12

1 实验背景

1.1 SM3 算法概述

SM3 是中国国家密码管理局于 2010 年发布的密码杂凑算法标准 (GM/T 0004-2012)，是我国商用密码体系中的重要组成部分。SM3 输出 256 位 (32 字节) 的杂凑值，广泛应用于数字签名、消息认证码、随机数生成等密码学场景。

SM3 算法核心结构：

SM3 采用 Merkle-Damgård 迭代结构，包含以下主要步骤：

- 消息填充 (Padding)：** 在消息末尾追加 bit “1”、若干 bit “0” 以及 64 位原始消息长度，使得填充后总长度为 512 的倍数。
- 消息扩展 (Message Expansion)：** 将每个 512 位消息块 B 扩展为 132 个 32 位字：
 - $W[0..15] = B$ 的分组
 - $W[j] = P_1(W[j-16] \oplus W[j-9] \oplus (W[j-3] \lll 15)) \oplus (W[j-13] \lll 7) \oplus W[j-6], j=16..67$
 - $W'[j] = W[j] \oplus W[j+4], j=0..63$
- 压缩函数 (Compression Function)：** 对每个消息块执行 64 轮迭代：
 - $SS1 = ((A \lll 12) + E + (T_j \lll (j \bmod 32))) \lll 7$
 - $SS2 = SS1 \oplus (A \lll 12)$
 - $TT1 = FF_j(A, B, C) + D + SS2 + W'_j$
 - $TT2 = GG_j(E, F, G) + H + SS1 + W_j$
 - 状态更新： $A \leftarrow TT1, B \leftarrow A, C \leftarrow B \lll 9, D \leftarrow C, E \leftarrow P_0(TT2), F \leftarrow E, G \leftarrow F \lll 19, H \leftarrow G$
- 输出：** 最终 8 个状态字经异或后输出为 256 位杂凑值。

1.2 SIMD 优化背景

传统的标量实现一次只能处理一条消息，无法充分利用现代 CPU 的 SIMD (Single Instruction Multiple Data) 并行计算能力。本实验分别针对 x86 架构 (AVX2 指令集, 256 位寄存器) 和 ARM64 架构 (NEON 指令集, 128 位寄存器) 实现了 SIMD 优化，同时采用 **SIMD 寄存器与通用寄存器混合** 的优化策略，兼顾并行效率与代码灵活性。

特性	x86 AVX2	ARM64 NEON
SIMD 寄存器宽度	256-bit (YMM)	128-bit (Q)
并行消息数	8 路 (8×32bit)	4 路 (4×32bit)
寄存器数量	16 个 YMM	32 个 Q
指令格式	二操作数	三操作数

2 实现方式

2.1 项目结构

```

1 lab2/
2     sm3.h           # 通用头文件
3     sm3_ref.c       # 标量参考实现（通用寄存器版本）
4     sm3_avx2.c      # x86 AVX2 SIMD优化（8路并行）
5     sm3_neon.c      # ARM64 NEON SIMD优化（4路并行）
6     sm3_test.c      # 测试程序
7     Makefile        # 编译构建系统

```

2.2 混合优化策略

本实验的核心创新点是 **SIMD 寄存器与通用寄存器的混合使用**：

SIMD 寄存器 (YMM/Q)	通用寄存器 (GPR)
8 路 (A~H) 状态并行更新	消息扩展标量计算
布尔函数 FFj/GGj	内存地址寻址
置换函数 P0/P1	循环控制
32 位加法、异或、循环移位	轮常量加载
轮常量广播	消息填充管理

为什么使用混合策略？

- 消息扩展适合通用寄存器**：消息扩展公式 $W[j] = P1(W[j-16] \oplus W[j-9] \oplus (W[j-3] \lll 15)) \oplus (W[j-13] \lll 7) \oplus W[j-6]$ 涉及跨索引的复杂依赖关系，不同 lane 的 W 值各不相同，使用标量指令逐消息计算反而更高效。
- 压缩轮函数适合 SIMD 寄存器**：64 轮压缩迭代中，每一轮对 8 个状态变量执行相同的移位、异或、加法操作。这些操作天然适合 SIMD 并行——在 AVX2 下一次处理 8 条消息，在 NEON 下一次处理 4 条消息。
- 寄存器分配优化**：现代 CPU 的 SIMD 和通用寄存器是独立的物理寄存器文件，混合使用可以最大化寄存器资源利用率，避免寄存器溢出 (spill) 到内存。

2.3 AVX2 SIMD 实现细节

消息布局

8 条独立消息的状态变量在 YMM 寄存器中的布局：

```

1 YMM寄存器为256位宽，由8个32位lane组成，每个lane保存一条独立消息的一个状态字：
2
3 | Lane 0 | Lane 1 | Lane 2 | Lane 3 | Lane 4 | Lane 5 | Lane 6 | Lane 7 |
4 |---|---|---|---|---|---|---|---|
5 | 32-bit | 32-bit | 32-bit | 32-bit | 32-bit | 32-bit | 32-bit | 32-bit |
6

```

7 8个YMM寄存器分别保存8条消息的A~H共8个状态变量。

关键 SIMD 操作实现

```
1  /* 8路32位循环左移（使用shift+or模拟，AVX2无原生rotate） */
2  static inline __m256i mm256_rotl32(__m256i x, int n) {
3      __m256i left = _mm256_slli_epi32(x, n); /* VPSLLD */
4      __m256i right = _mm256_srli_epi32(x, 32 - n); /* VPSRLD */
5      return _mm256_or_si256(left, right);      /* VPOR */
6  }
7
8  /* FF_j(X,Y,Z): 前16轮为XOR, 后48轮为多数函数 */
9  static inline __m256i mm256_FF(int j, __m256i x, __m256i y, __m256i z) {
10     if (j < 16)
11         return _mm256_xor_si256(_mm256_xor_si256(x, y), z); /* VPXOR */
12     else {
13         /* (X AND Y) OR (X AND Z) OR (Y AND Z) */
14         __m256i xy = _mm256_and_si256(x, y); /* VPAND */
15         __m256i xz = _mm256_and_si256(x, z);
16         __m256i yz = _mm256_and_si256(y, z);
17         return _mm256_or_si256(_mm256_or_si256(xy, xz), yz); /* VPOR */
18     }
19 }
```

一轮压缩的 SIMD 实现

```
1  for (j = 0; j < 64; j++) {
2      ymm_A_rot12 = mm256_rotl32(ymm_A, 12);
3      ymm_Tj      = _mm256_set1_epi32(SCALAR_ROT132(get_Tj(j), j & 31));
4      ymm_SS1     = mm256_rotl32(
5          _mm256_add_epi32(
6              _mm256_add_epi32(ymm_A_rot12, ymm_E), ymm_Tj), 7);
7      ...
8      ymm_E = mm256_P0(ymm_TT2);
9      // 状态旋转通过YMM寄存器间重命名实现
10 }
```

2.4 ARM64 NEON 实现细节

ARM64 NEON 使用 128 位 Q 寄存器，每个寄存器容纳 4 个 32 位 lane，一次处理 4 条消息。

ARM64 NEON 的优势：

- 三操作数指令：vaddq_u32(dst, src1, src2) —减少 MOV 操作
- VBIC 指令：vbicq_u32(z, x) 一条指令实现 Z AND NOT X，GG 函数更高效
- 32 个 128 位寄存器：寄存器资源充裕，减少 spill
- VSHL/VSLI：移位指令丰富

```
1  /* GG_j: NEON的VBIC指令天然支持 NOT X AND Z */
2  static inline uint32x4_t neon_GG(int j, uint32x4_t x,
3                                     uint32x4_t y, uint32x4_t z) {
4      if (j < 16)
5          return veorq_u32(veorq_u32(x, y), z);
6      else {
7          uint32x4_t xy = vandq_u32(x, y);
8          uint32x4_t nxz = vbicq_u32(z, x); // NEON专有: Z AND NOT X
9          return vorrq_u32(xy, nxz);
10     }
11 }
```

2.5 关键技术处理

1. 移位未定义行为处理

C 语言标准规定，对 32 位整数移位 ≥ 32 位是未定义行为。SM3 算法中轮常量计算涉及 $\text{ROTL32}(T_j, j)$ 其中 $j = 0..63$ ，当 $j \geq 32$ 时触发 UB。

解决方案：使用 $j \& 31$ （等效于 $j \bmod 32$ ）替代 j ，因为 32 位循环移位在模 32 下等价：

```
1  /* 修正前 */
2  ROTL32(T_j, j);    /* j>=32 时UB */
3
4  /* 修正后 */
5  ROTL32(T_j, j & 31); /* j&31 < 32, 安全 */
```

2. 大端序处理

SM3 标准使用 big-endian 字节序。本实现中：

- `bytes_to_words()`: 将大端字节数组转为 32 位主机序字
- `words_to_bytes()`: 将 32 位主机序字转回大端字节数组
- 消息长度字段始终按大端序写入

3 实现过程

3.1 开发步骤

整个开发过程分为四个阶段推进。

阶段零：项目结构搭建

在动手写代码之前，首先设计好项目的模块划分和接口约定。所有实现共享同一份头文件 `sm3.h`，确保 API 一致、可互换调用。

项目文件清单参见上文 2.1 节项目结构，具体代码见各源文件。

项目分为六层：

- 公共头文件 (sm3.h): 定义上下文结构体 sm3_ctx_t 和统一 API
- 标量实现 (sm3_ref.c): 纯 C 语言 + 通用寄存器, 作为功能正确性基准
- AVX2 实现 (sm3_avx2.c): 8 路并行 SIMD, 核心优化版本
- NEON 实现 (sm3_neon.c): 4 路并行 SIMD, ARM64 适配版本
- 测试程序 (sm3_test.c): 覆盖 4 类测试
- 构建系统 (Makefile): 支持 x86/ARM64 多目标

关键设计决策: 三个实现文件各自独立编译, 通过统一的 sm3.h 头文件暴露接口; 测试程序同时链接标量实现和 SIMD 实现, 直接对比两者的输出结果。

第一阶段: 标量参考实现 (sm3_ref.c)

整个实验的基础, 先有一个 100% 正确的标量版本, 后续 SIMD 优化才能做交叉验证。

步骤 1-1: 研读标准

研读 GM/T 0004-2012 标准文档, 重点理解:

- 消息填充规则 (追加 “1” → 补零 → 追加 64 位长度)
- 消息扩展公式中 $W[16..67]$ 的递推关系
- 压缩函数 64 轮的位运算逻辑
- 初始向量 IV 的 8 个 32 位常量

步骤 1-2: 实现压缩函数

SM3 的核心是压缩函数 sm3_compress(V, B), 它将 256 位链接变量 V 和 512 位消息块 B 压缩为新的 256 位状态。这是后续所有优化工作的起点。

完整实现参见 sm3_ref.c 中的 sm3_compress() 函数。

具体实现的要点:

1. **消息扩展** (函数 sm3_message_expansion): 先将 16 个字直接复制为 $W[0..15]$, 再按公式递推出 $W[16..67]$ 。注意 W 数组需声明为 68 个元素 (0..67), 因为 $W'[j] = W[j] \oplus W[j+4]$ 在 $j=63$ 时需要访问 $W[67]$ 。
2. **64 轮迭代**: 每轮使用相同的操作模板 (移位 → 异或 → 加法), 但根据轮序号切换 Tj 常量 (前 16 轮 0x79CC4519, 后 48 轮 0x7A879D8A) 和 FF/GG 函数 (前 16 轮 XOR, 后 48 轮多数/选择函数)。
3. **状态更新**: 采用原地旋转策略——A/B/C/D 各移一位、E/F/G/H 各移一位。这是 Merkle-Damgard 结构的经典模式, 与 SHA-256 的状态更新类似。
4. **Merkle-Damgard 前馈**: 压缩函数结束时将新旧状态异或, 这是防碰撞的关键设计。

步骤 1-3: 实现增量 API

除了便捷的一次性函数 sm3_hash(), 还实现了流式接口:

- `sm3_init()` —初始化上下文，设置 IV
- `sm3_update()` —追加数据，内部缓冲不满一块的字节
- `sm3_final()` —填充并处理最后一块，输出 256 位摘要

增量 API 的关键难点在于消息填充逻辑：必须正确处理“数据刚好占满一整块”和“不足一块”两种边界情况。具体而言，当数据刚好为 64 字节时，`sm3_update` 处理完该块后缓冲区为空，`sm3_final` 只需追加填充块；当数据不足 64 字节时，`sm3_final` 在一个块内同时完成数据 + 填充。

步骤 1-4：标准向量验证

使用 GM/T 0004-2012 附录 A 的三组标准测试向量验证正确性。这一步在开发中实际发现了一个隐蔽 bug：`ROTL32(T_j, j)` 在 $j \geq 32$ 时触发 C 语言未定义行为（详见第四章 4.1 节），修正为 `ROTL32(T_j, j & 31)` 后全部通过。

第二阶段：x86 AVX2 SIMD 优化 (`sm3_avx2.c`)

这是本实验的核心工作。目标是在不改变算法输出的前提下，利用 AVX2 的 256 位 SIMD 寄存器同时处理 8 条独立消息。

步骤 2-1：设计 8 路并行数据布局

每条 SM3 消息的压缩状态是 8 个 32 位字 (A~H)。AVX2 的 YMM 寄存器为 256 位 = 8×32 位，因此一个 YMM 寄存器恰好容纳 8 条消息的同一个状态字：

```
1 ymm_A = [msg0.A | msg1.A | msg2.A | ... | msg7.A] (每条消息的A值在一个lane中)
2 ymm_B = [msg0.B | msg1.B | ...                ] (B值)
3 ...
4 ymm_H = [msg0.H | msg1.H | ...                ] (H值)
```

8 个 YMM 寄存器刚好存下 8 条消息的全部状态，无需额外的 load/store。

步骤 2-2：实现 SIMD 版本的位操作函数

AVX2 没有原生的 32 位循环移位指令，必须用左移 + 右移 + 或来模拟。对于 8 路并行的 FF/GG 布尔函数，每条 lane 独立运算相同的逻辑，使用 `_mm256_and_si256` / `_mm256_xor_si256` / `_mm256_or_si256` 即可实现。

步骤 2-3：实现 8 路并行压缩函数

这是整个优化中最复杂的部分：

完整实现参见 `sm3_avx2.c` 中的 `sm3_compress_x8()` 函数。

每轮压缩中，所有 32 位加法、异或、旋转都在 8 个 lane 上同步执行。轮常量用 `_mm256_set1_epi32()` 广播到全部 8 个 lane。消息字 `W[j]` 和 `W'[j]` 则从内存中逐 lane 收集（标量地址计算 + SIMD 加载）。

步骤 2-4：混合策略——消息扩展仍用标量

消息扩展公式中的 P1 置换存在跨索引依赖 (`W[j]` 依赖 `W[j-16]`、`W[j-9]`、`W[j-3]` 等)，不同消息的 W 值完全不同，使用 SIMD 没有优势。因此消息扩展保持逐消息标量计算，这也是“SIMD+通用寄存器混合”策略的关键体现。

步骤 2-5：交叉验证

编写测试代码将 AVX2 8 路并行输出与标量逐条计算输出逐一对比，确保所有 lane 的结果完全一致（见 3.2 节 Test 3）。

第三阶段：ARM64 NEON SIMD 优化 (sm3_neon.c)

ARM64 的 NEON 指令集使用 128 位 Q 寄存器 (4×32 位)，一次处理 4 条消息。

实现过程与 AVX2 类似，但有几点差异：

1. **寄存器资源更充裕**：ARM64 有 32 个 128 位 NEON 寄存器 (vs x86 的 16 个 YMM)，临时变量可以保留在寄存器中而不必 spill 到栈上。
2. **三操作数指令**：ARM 指令格式 `vaddq_u32(dst, src1, src2)` 比 x86 的 `dst = dst + src` 更灵活，减少了一次 MOV 操作。
3. **VBIC 位清除指令**：NEON 的 `vbicq_u32(z, x)` 可直接计算 `Z AND NOT X`，在 GG 函数中一条指令替代了 x86 的反转 + 与操作。
4. **交叉编译**：由于开发环境为 x86，ARM64 版本通过 `aarch64-linux-gnu-gcc` 交叉编译器构建验证 (Makefile 中的 `arm64` 目标)。

第四阶段：测试与性能评估

最后阶段构建了完整的测试体系：

1. **标准测试向量**：确保标量实现的算法正确性
2. **增量 API 测试**：验证流式接口的块缓冲逻辑
3. **交叉验证**：证明 AVX2 SIMD 实现的功能等价性
4. **性能基准**：对比 4 种消息大小下标量与 SIMD 的吞吐量

最终的编译和测试流程只需一条命令 `mingw32-make`：

构建规则参见 Makefile，测试代码参见 `sm3_test.c`。

编译参数 `-O3 -mavx2 -mfma -D__AVX2__` 启用了最高级别优化和 AVX2 指令集支持。

3.2 测试结果

运行 `mingw32-make` 后，全部四项测试均通过，具体结果如下。

Test 1: 标准测试向量验证

测试了 GM/T 0004-2012 附录 A 中规定的三组标准测试向量，验证标量参考实现的正确性。

```
SM3 Test 1 - Standard Test Vectors
=== Test 1: Standard Test Vectors ===

[PASS] SM3("abc")
digest: 66c7f0f462eedd9d1f2d46bdc10e4e2
       4167c4875cf2f7a2297da02b8f4ba8e0

[PASS] SM3("abcdabcdabcdabcdabcdabcdabcdabcd")
digest: debe9ff92275b8a138604889c18e5a4d
       6fdb70e5387e5765293dcba39c0c5732

[PASS] SM3(empty message)
digest: 1ab21d8355cfa17f8e61194831e81a8f
       22bec8c728fefb747ed035eb5082aa2b

Result: 3 passed, 0 failed
```

图 1: 标准测试向量验证

三组测试向量全部通过:

- SM3("abc") 与标准向量完全一致
- SM3("abcd"×16) 与标准向量完全一致
- SM3(空消息) 与标准向量完全一致

这证明标量参考实现严格遵循 GM/T 0004-2012 标准, 消息填充、消息扩展、压缩函数三个核心环节均正确。

Test 2: 增量 API 测试

验证分块调用 sm3_update() 多次更新与一次性调用 sm3_hash() 的结果一致性。

```
SM3 Test 2 - Incremental API Test
=== Test 2: Incremental API Test ===

[PASS] Incremental API matches one-shot API
digest: ae7b699cc6041487ff8de53d85f221c
       20bea09d90fc3adea482c2ec05e217f9d

Result: 0 failed
```

图 2: 增量 API 测试

增量 API (init/update/final) 与一次性 API 输出完全一致, 证明上下文管理、消息缓冲、块边界处理逻辑正确。

Test 3: 交叉验证——标量 vs AVX2 SIMD

这是最关键的验证, 确保 AVX2 SIMD 优化版本的 8 路并行计算结果与标量参考实现逐条计算的结果完全一致。

```
SMB Test 3 - Cross-Validation (Scalar vs AVX2 SIMD)
=== Test 3: Cross-Validation (Scalar vs AVX2 SIMD) ===

[PASS] Lane 0 matches reference
[PASS] Lane 1 matches reference
[PASS] Lane 2 matches reference
[PASS] Lane 3 matches reference
[PASS] Lane 4 matches reference
[PASS] Lane 5 matches reference
[PASS] Lane 6 matches reference
[PASS] Lane 7 matches reference
[PASS] Different messages - Lane 0 matches
[PASS] Different messages - Lane 1 matches
[PASS] Different messages - Lane 2 matches
[PASS] Different messages - Lane 3 matches
[PASS] Different messages - Lane 4 matches
[PASS] Different messages - Lane 5 matches
[PASS] Different messages - Lane 6 matches
[PASS] Different messages - Lane 7 matches

Result: 16 passed, 0 failed
```

图 3: 交叉验证

全部 16 项交叉验证通过，涵盖两种场景：

- **相同消息 ×8 路**：8 条完全相同的消息同时计算，每个 lane 的 SIMD 输出与标量参考一致
- **不同消息 ×8 路**：8 条内容不同但长度相同的消息，每个 lane 的输出与对应的标量参考一致

这证明 8 路并行布局和 SIMD 压缩函数实现是正确的。

Test 4: 性能基准测试

测试环境：x86-64, GCC 8.1 (MinGW-W64), -O3 -mavx2 -mfma 优化。

```
SMB Test 4 - Performance Benchmark
=== Test 4: Performance Benchmark ===

[Scalar]   64 B x 10000   3447.30 us   177.05 MB/s
[AVX2 ]   64 B x 10000  24415.00 us   199.99 MB/s

[Scalar]  1024 B x 1000   3046.40 us   320.56 MB/s
[AVX2 ]  1024 B x 1000  18227.40 us   428.61 MB/s

[Scalar] 10240 B x 100    2629.70 us   371.36 MB/s
[AVX2 ] 10240 B x 100   23045.70 us   339.00 MB/s

[Scalar] 102400 B x 10    2648.50 us   368.72 MB/s
[AVX2 ] 102400 B x 10   19793.10 us   394.71 MB/s

ALL TESTS PASSED
```

图 4: 性能基准测试

消息大小	标量吞吐量	AVX2 SIMD 吞吐量 (8 路)	加速比
64 B	177.05 MB/s	199.99 MB/s	1.13x
1 KB	320.56 MB/s	428.61 MB/s	1.34x
10 KB	371.36 MB/s	339.00 MB/s	0.91x
100 KB	368.72 MB/s	394.71 MB/s	1.07x

性能分析：

1. **小消息 (64B)**：SIMD 版本略有加速，但由于 8 路数组的初始化、消息扩展结果提取等开销，加速比有限。若仅计算单条消息，8 路并行方案会浪费 7 个 lane 的计算能力，因此该模式更适合批量消息场景。
2. **中等消息 (1KB)**：SIMD 版本加速最明显 (1.34x)，此时消息扩展的计算量增加，压缩函数的 SIMD 并行优势开始覆盖固定开销。
3. **大消息 (10-100KB)**：吞吐量趋于稳定，标量和 SIMD 的差距缩小。SIMD 版本在 100KB 时仍有 1.07x 加速，但 10KB 时反而略低——这是因为内存带宽成为瓶颈，8 路并行需要同时加载 8 条消息的数据，cache 竞争加剧。
4. **适用场景**：SIMD 版本最适合**批量消息处理**场景——如服务器端同时为多个客户端计算 SM3 签名、区块链批量验签等。单条消息场景建议使用标量实现。

4 实验中遇到的问题与解决

4.1 循环移位未定义行为

问题：SM3 标准中轮常量使用 $T_j \lll j$ ，其中 j 从 0 到 63。C 语言中 32 位整数移位 ≥ 32 位是 UB，导致 GCC 在优化编译时产生错误的结果。

发现过程：标量实现的“abc”和空消息测试通过，但 64 字节测试失败。通过分析发现，当 $j \geq 32$ 时编译期常量折叠与运行时 x86 移位行为不一致。

解决：使用 $j \& 31$ 限制移位量，保证正确性。

4.2 不同消息长度导致 SIMD 结果不匹配

问题：测试中 8 路并行要求所有消息长度一致，初始测试代码使用了不同长度的消息，导致 hash 结果不匹配。

解决：修改测试用例，确保所有 8 条消息长度完全一致。

4.3 测试向量中 hex 字符串长度计算

问题：64 字节“abcd”测试向量使用了 hex 字符串，在代码中跨行拼接时可能存在不可见字符或长度偏差。

解决：改用直接内存构造方式生成 64 字节测试消息，彻底排除 hex 解析因素。

5 总结

本实验完成了 SM3 密码杂凑算法的 SIMD 优化实现，涵盖以下内容：

1. **标量参考实现**：严格遵循 GM/T 0004-2012 标准，通过了全部标准测试向量验证。
2. **x86 AVX2 优化**：实现了 8 路并行 SM3 哈希，利用 256 位 YMM 寄存器同时对 8 条消息进行压缩迭代，在批量处理场景下有效提升吞吐量。
3. **ARM64 NEON 优化**：实现了 4 路并行 SM3 哈希，利用 128 位 Q 寄存器同时对 4 条消息进行处理，适配 ARM64 移动和服务平台。
4. **混合寄存器策略**：创新性地使用 SIMD 寄存器处理压缩轮函数的并行状态更新，使用通用寄存器处理消息扩展的标量计算，充分发挥了两种寄存器类型的优势。
5. **完整的测试体系**：包含标准向量验证、交叉验证、增量 API 测试和性能基准。

本实验加深了对 SM3 算法原理、SIMD 并行编程、不同 CPU 架构指令集差异的理解，为后续在密码学算法优化方面的研究和工程实践奠定了基础。

参考文献

1. GM/T 0004-2012 《SM3 密码杂凑算法》，国家密码管理局，2012
2. GB/T 32905-2016 《信息安全技术 SM3 密码杂凑算法》，国家标准化管理委员会，2016

6 分工情况

姓名 (学号)	具体分工内容
杨阳 (202300460010)	项目架构设计、标量参考实现、算法正确性验证 • 设计项目结构和 API 接口 • 实现 <code>sm3_ref.c</code> 标量 SM3 (含消息填充、消息扩展、压缩函数) • 编写标准测试向量验证程序 • 调试并修复循环移位 UB 问题 • 撰写实验报告 (背景、实现方式)
孙琬竺 (202300460020)	x86 AVX2 SIMD 优化实现 • 设计 8 路并行消息布局 (YMM 寄存器分配) • 实现 <code>sm3_avx2.c</code> (含 SIMD 版本的 $P_0/P_1/FF/GG$ 函数) • 实现 8 路并行压缩函数和消息扩展 • 性能基准测试与优化 • 撰写实验报告 (AVX2 实现细节)
胡子玮 (202300460032)	ARM64 NEON SIMD 优化实现 • 设计 4 路并行消息布局 (Q 寄存器分配) • 实现 <code>sm3_neon.c</code> (含 NEON 专有指令优化) • ARM64 交叉编译配置 • 标量与 SIMD 交叉验证测试 • 撰写实验报告 (NEON 实现细节)
熊芷萱 (202300460163)	测试程序、构建系统、文档撰写 • 编写 <code>sm3_test.c</code> (测试向量、交叉验证、性能基准) • 编写 Makefile (支持 x86/ARM64/交叉编译多目标) • 测试结果汇总与性能数据分析 • 整体文档整合与实验报告排版 • 撰写实验报告 (实现过程、问题总结)